

01 · AND / OR / XOR

템플릿에서 비트스트림까지

VS Code 사전 시뮬레이션 → Vivado 2026.1

1. 공개 템플릿을 clone하고 새 창에서 연다.
2. 네 입력 조합을 검사하고 파형을 읽는다.
3. 같은 소스를 Vivado에 등록하고 다시 시뮬레이션한다.
4. Spartan-7용 비트스트림을 생성한다.

작성일 2026. 09. 10. 이해리

01 · 실습 목차

1부 · VS Code

도구 확인·템플릿 clone·새 창 열기

RTL·테스트벤치·예상값

실행 task·로그·VaporView·실험 전 레포트

2부 · Vivado 2026.1

프로젝트 생성·소스 임포트·top 지정

Behavioral Simulation·파형 비교

핀 확인·합성·구현·비트스트림

좌하단 **목차**는 이 PDF의 2쪽으로 돌아온다.

이번 검토본의 끝은 비트스트림 생성이다. 실제 보드 기록은 다음 실험 단계다.



1. 사용할 프로그램을 구분한다

프로그램	이번 실습에서 하는 일
Git	공개 저장소의 템플릿과 소스를 내려받는다.
Python 3	workspace의 공통 실행 스크립트를 실행한다.
VS Code	코드 편집·작업 실행·로그 열기.
slang-server	Verilog/SystemVerilog 구문 강조와 진단.
VaporView	VCD 파일을 파형으로 표시.
Vivado 2026.1	XSim, FPGA 합성·배치배선·비트스트림 생성.

VS Code 자체가 시뮬레이터는 아니다. Vivado GUI를 열기 전에 XSim을 실행하는 구성이다. Vivado 도구는 사전에 설치되어 있어야 한다.

Vivado 설치 PDF VS Code 환경설정 PDF

2. 버전이 출력되는지 먼저 확인

PowerShell을 열고 한 줄씩 실행한다.

<code>git --version</code>	Git 버전
<code>python --version</code>	Python 3.x
<code>code --version</code>	VS Code 버전
<code>vivado -version</code>	Vivado 2026.1
<code>xvlog --version</code>	컴파일 도구
<code>xelab --version</code>	연결·시뮬레이션 모델 생성
<code>xsim --version</code>	시뮬레이션 실행 도구

vivado는 `-version`, 나머지 XSim 도구는 `--version`이다.

이번 실행에서 Git 2.55.0, VS Code 1.136.1, XSim 2026.1을 확인했다.

3. 명령을 찾지 못하면

1. Git·Python·VS Code가 없으면 해당 프로그램부터 설치한다. Python은 설치 시 Add python.exe to PATH를 선택한다.
2. Vivado가 없다면 설치 매뉴얼을 따른다. 설치했는데 도구가 안 잡히면 설치 폴더의 bin을 확인한다.
3. Windows 환경 변수 VIVADO_BIN을 그 bin 폴더로 지정하거나 PATH에 추가한다.
4. VS Code를 완전히 닫고 다시 연 뒤 버전을 재확인한다.

설치 경로 예: C:/AMDDesignTools/2026.1/Vivado/bin.

PC마다 다르므로 경로를 그대로 외우지 않는다. 버전 출력이 있어야 다음으로 간다.

4. 공개 템플릿 clone

실습 폴더를 둘 위치에서 아래 명령을 실행한다. `git clone --depth 1`

```
--filter=blob:none --sparse  
--branch daily/0910  
https://github.com/Glaysia/ece2_26_2_2.git lab1
```

위 세 줄은 한 명령이다. README의 한 줄 명령을 복사해 실행한다.
daily/0910은 이번 검토본 브랜치다.

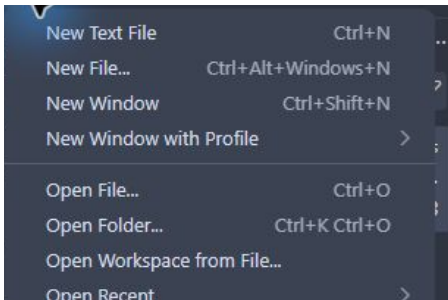
```
cd lab1  
git sparse-checkout set example/fpga_projects_hdl/LAB1
```

과거 교안 전체를 받지 않고 LAB1 실행 자료만 펼친다.
이미 lab1 폴더가 있으면 다른 폴더명을 사용한다.

▶ 설치·clone 명령 원문 열기

5. VS Code 새 창과 workspace

File → New Window를 누른다. 새 창의 File → Open Workspace from File...을 선택한다.

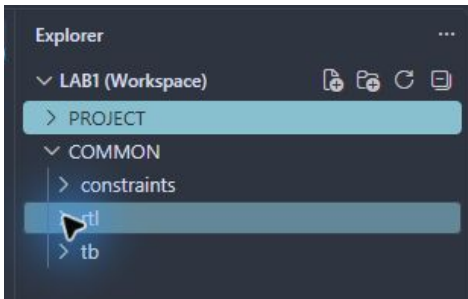


단축키 Ctrl+Shift+N은 새 창 열기다. Open File...로 코드 하나만 여는 것과 구분한다.

6. 항상 같은 템플릿을 연다

clone한 폴더에서 아래 workspace를 선택한다.

example/fpga_projects_hdl/LAB1/template/LAB1.code-workspace



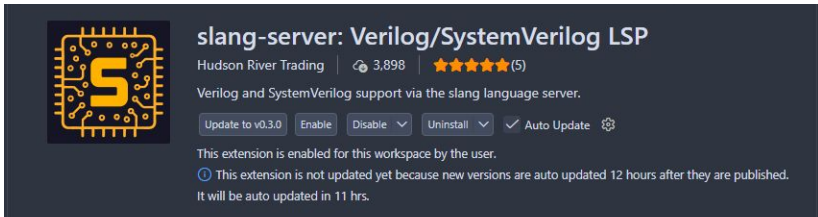
PROJECT: 실행 설정·결과·Vivado 프로젝트.

COMMON: 공통 RTL·테스트벤치·핀 제약.

이 템플릿은 최신 첫 회로를 가리킨다.

7. Verilog 확장을 켜다

왼쪽 Extensions → @installed slang 검색 → 상세 화면.



Enable → Enable (Workspace). 이미 켜져 있으면 그대로 둔다.
없으면 hudson-river-trading.vscode-slang을 설치한다.
같은 방법으로 lramseyer.vaporview도 설치·활성화한다.

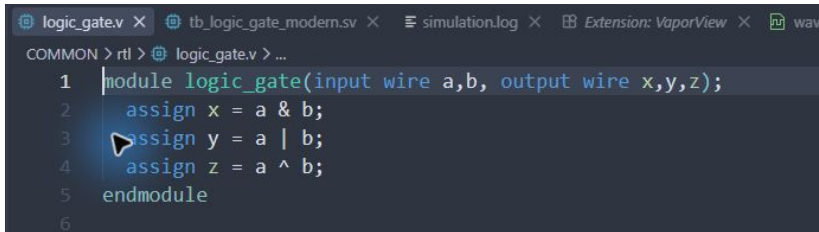
8. 실행 전 예상값

a	b	x AND	y OR	z XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

이번 최신 테스트는 $00 \rightarrow 01 \rightarrow 10 \rightarrow 11$, 각 10ns다.
OR는 하나 이상 1, XOR는 서로 다른 입력일 때 1이다.
이 표를 실험 전 레포트에 먼저 작성한다.

9. RTL 파일을 읽는다

Explorer에서 COMMON → rtl → logic_gate.v를 두 번 누른다.



```
COMMON > rtl > logic_gate.v > ...  
1 module logic_gate(input wire a,b, output wire x,y,z);  
2     assign x = a & b;  
3     assign y = a | b;  
4     assign z = a ^ b;  
5 endmodule  
6
```

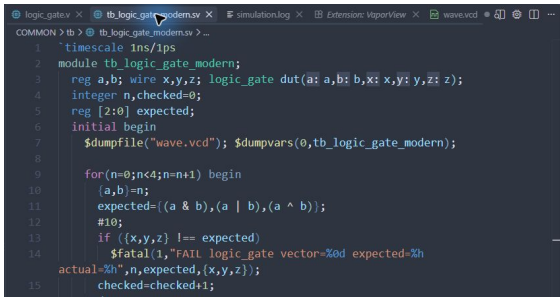
입력 a·b와 출력 x·y·z를 확인한다.

assign 세 줄이 AND·OR·XOR 조합회로를 연결한다.

슬라이드의 소스는 실제 Nord VS Code 캡처다. 실행 파일은 저장소에 있다.

10. 테스트벤치의 역할

COMMON → tb → tb_logic_gate_modern.sv를 연다.



```
COMMON > tb > tb_logic_gate_modern.sv > ...
1 `timescale 1ns/1ps
2 module tb_logic_gate_modern;
3   reg a,b; wire x,y,z; logic_gate dut(a: a,b: b,x: x,y: y,z: z);
4   integer n,checked=0;
5   reg [2:0] expected;
6   initial begin
7     $dumpfile("wave.vcd"); $dumpvars(0,tb_logic_gate_modern);
8
9     for(n=0;n<4;n=n+1) begin
10       {a,b}=n;
11       expected=((a & b),(a | b),(a ^ b));
12       #10;
13       if ({x,y,z} != expected)
14         $fatal(1,"FAIL logic_gate vector=%0d expected=%h
15       actual=%h",n,expected,{x,y,z});
16       checked=checked+1;
17     end
18   end
19 endmodule
```

dut는 검사할 논리 게이트다. {a,b}=n으로 네 입력을 만든다.
expected와 실제 $x \cdot y \cdot z$ 가 다르면 \$fatal로 실패한다.
확장 진단에 오류가 없다는 것과 시뮬레이션 통과는 다르다.

11. 시간·완료 조건 확인

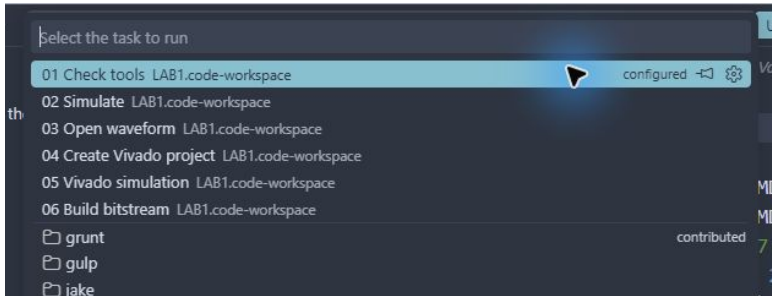
- timescale 1ns/1ps: 시간 단위는 1ns, 정밀도는 1ps.
- #10: 입력을 준 뒤 10ns 동안 기다린다.
- 4개를 모두 검사하면 LAB1_PASS logic_gate cases=4.
- \$finish로 40ns에서 종료한다.
- wave.vcd: 시뮬레이션 파형 원본.

기존 레거시 교안의 200ns 간격과 구분한다.

최신 VS Code와 최신 Vivado는 이 동일한 테스트벤치를 사용한다.

12. 저장하고 작업 메뉴 열기

코드를 수정했다면 File → Save All. 다음은 Terminal → Run Task....



창이 좁으면 상단 ...에 Terminal 메뉴가 있다.
먼저 01 Check tools, 다음에 02 Simulate를 선택한다.
Ctrl+Shift+B 기본 작업도 02 Simulate다.

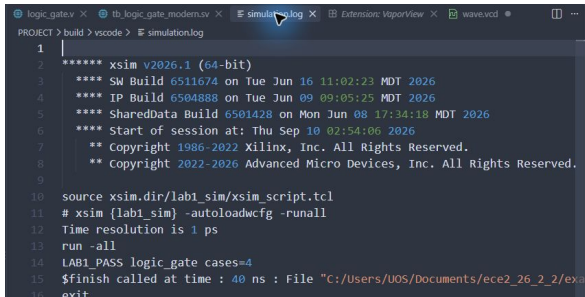
13. 여섯 작업의 의미

01 Check tools	설치·버전 검사
02 Simulate	VS Code에서 XSim 실행
03 Open waveform	생성한 VCD를 VS Code로 열기
04 Create Vivado project	소스·테스트벤치·제약을 XPR에 등록
05 Vivado simulation	Vivado 프로젝트 기반 재검사
06 Build bitstream	합성·배치배선·비트스트림 생성

작업이 보이지 않으면 LAB1.code-workspace를 열었는지 확인한다.
실행 스크립트는 공통 tools/lab1.py, 회로 설정은 project.json이다.

14. PASS와 종료 시각 확인

PROJECT → build → vscode → simulation.log를 연다.



```
1 |
2 ***** xsim v2026.1 (64-bit)
3 **** SW Build 6511674 on Tue Jun 16 11:02:23 MDT 2026
4 **** IP Build 6504888 on Tue Jun 09 09:05:25 MDT 2026
5 **** SharedData Build 6501428 on Mon Jun 08 17:34:18 MDT 2026
6 **** Start of session at: Thu Sep 10 02:54:06 2026
7 ** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
8 ** Copyright 2022-2026 Advanced Micro Devices, Inc. All Rights Reserved.
9
10 source xsim.dir/lab1_sim/xsim_script.tcl
11 # xsim {lab1_sim} -autoloadwcfg -runall
12 Time resolution is 1 ps
13 run -all
14 LAB1_PASS logic_gate cases=4
15 $finish called at time : 40 ns : File "C:/Users/UOS/Documents/ece2_26_2_2/exa
16 exit
```

LAB1_PASS logic_gate cases=4와 40ns 종료를 확인한다.
실패하면 이번 실행의 compile.log·elaborate.log·simulation.log를
읽는다.
실행기는 실패 후 예전 VCD를 새 성공 결과로 남기지 않는다.

15. VCD를 VaporView로 열기

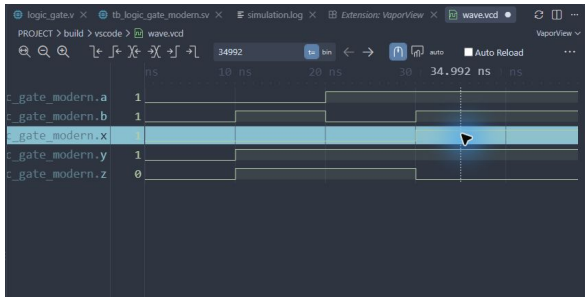
03 Open waveform을 실행하거나 build/vscode/wave.vcd를 연다.

글자로 열리면 파일 탭을 마우스 오른쪽 클릭 →
Reopen Editor With... → VaporView.

1. 빈 파형 화면의 Netlist View를 누른다.
2. tb_logic_gate_modern을 펼친다.
3. a, b, x, y, z를 각각 두 번 눌러 추가한다.
4. 파형 도구 모음의 Zoom to Fit로 전체 시간을 맞춘다.
5. 아래 Time Units를 ns로 맞춘다.

16. 파형에서 실제 값을 읽는다

시간 구간 안쪽을 클릭하여 세로 커서와 신호 값을 확인한다.



0-10ns: 00 / 10-20ns: 01 / 20-30ns: 10 / 30-40ns: 11.

마지막 구간에서 AND=1, OR=1, XOR=0인지 확인한다.

입력이 전환되는 경계 대신 구간의 중간을 읽는다.

17. 실험 전 레포트

1. 회로 목적·입출력·진리표·RTL 세 연결을 설명한다.
2. 템플릿 경로, 도구 버전, 실행 task, 코드 커밋을 기록한다.
3. 테스트벤치의 네 입력·기대값·10ns 간격·40ns 종료를 설명한다.
4. 자신의 PASS 로그와 $a \cdot b \cdot x \cdot y \cdot z$ 파형을 캡처한다.
5. 네 시간 구간을 표로 정리하고 OR와 XOR 차이를 해석한다.
6. 오류가 있었다면 원인·수정·저장·재실행 결과를 적는다.

저장소 소스를 복사해서 사용해도 된다. 구조와 결과를 설명해야 한다.
교재 캡처를 자신의 실행 결과로 제출하지 않는다.

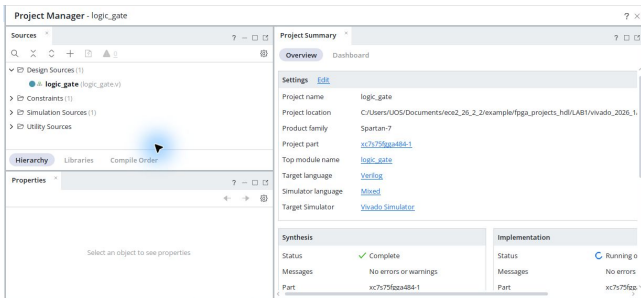
18. 이제 Vivado로 넘어간다

사전 레포트까지 작성했으면 04 Create Vivado project를 실행한다.

- 현재 템플릿의 RTL·테스트벤치·XDC를 같은 프로젝트에 등록한다.
- part는 xc7s75fgga484-1, 설계 top은 logic_gate.
- 생성 위치: PROJECT → vivado/logic_gate.xpr.
- Vivado 2026.1 → File → Project → Open...에서 이 XPR을 연다.

이 작업은 재생성용 Tcl을 사용한다. 같은 작업을 다시 실행하면 생성 프로젝트가 다시 만들어지므로, 직접 바꾼 설정은 project.json·소스·제약에도 반영한다.

19. 프로젝트에서 확인할 세 가지



Product family: Spartan-7, Project part: xc7s75fpga484-1,
Top module name: logic_gate.

Sources의 Design Sources·Simulation Sources·Constraints를 구분한다.

20. 직접 임포트하는 경우

새 프로젝트를 직접 만들 때는 File → Project → New...를 선택한다.

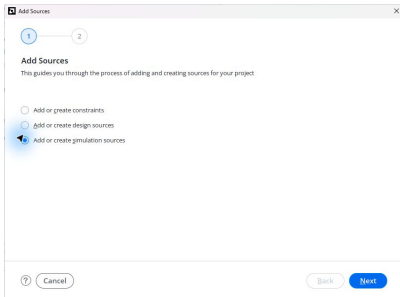
1. Project name: logic_gate, 위치: 자신의 실습 폴더.
2. RTL Project를 선택한다. part는 xc7s75fgga484-1.
3. 프로젝트를 열고 왼쪽 Add Sources를 누른다.
4. RTL은 Add or create design sources.
5. 테스트벤치는 Add or create simulation sources.
6. 핀 XDC는 Add or create constraints.

템플릿 task로 생성했다면 이미 등록되어 있다. 같은 파일을 중복 추가하지 않는다.

다음 화면은 실제 테스트벤치를 임포트하는 절차다.

21. RTL과 테스트벤치 구분

Add Sources에서 추가할 파일의 종류를 고른 뒤 Next.



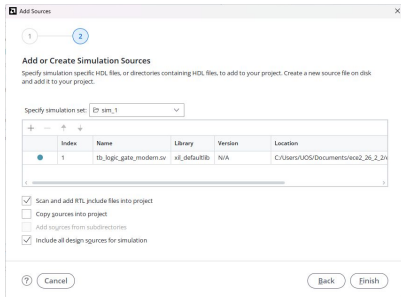
RTL: `common/rtl/logic_gate.v`

시뮬레이션: `common/tb/tb_logic_gate_modern.sv`

테스트벤치를 합성할 회로 top으로 지정하지 않는다.

22. Add Files와 Copy 옵션

Add Files → 같은 테스트벤치 선택 → Open → Finish.



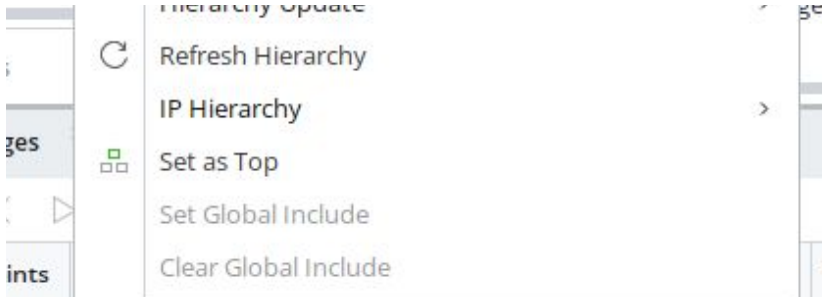
Copy sources into project는 해제하여 공유 원본을 참조한다.

Include all design sources for simulation은 체크한다.

이후 VS Code에서 원본을 수정하면 Vivado도 같은 파일을 사용한다.

23. 시뮬레이션 top 지정

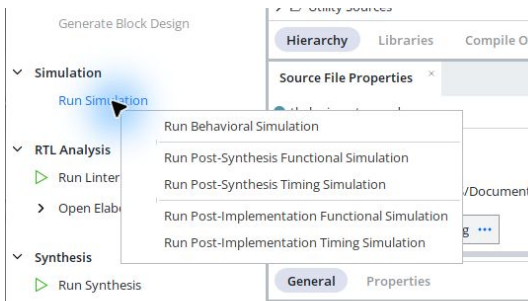
Sources → Simulation Sources → sim_1을 펼친다.



tb_logic_gate_modern 우클릭 → Set as Top.
Design Sources의 top은 logic_gate로 유지한다.
굵게 표시된 시뮬레이션 top 이름을 확인한다.

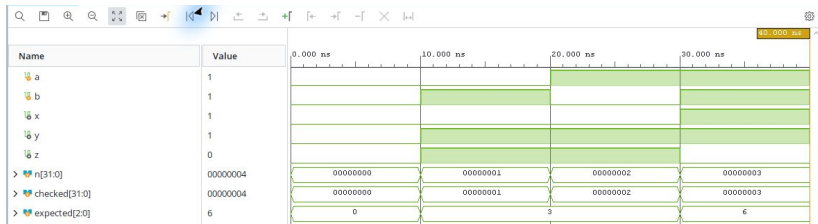
24. Vivado 시뮬레이션 실행

Flow Navigator → Run Simulation → Run Behavioral Simulation.



컴파일·elaboration이 끝나면 시뮬레이션 창이 열린다.
템플릿의 05 Vivado simulation도 같은 프로젝트에서 같은 검사를
실행한다.

25. Vivado 파형을 간단히 비교



파형에서 $a \cdot b \cdot x \cdot y \cdot z$ 를 확인하고 Zoom Fit로 0-40ns를 본다.
Tcl Console의 LAB1_PASS logic_gate cases=4와 종료 시각을 확인한다.
VS Code와 네 구간의 값이 같으면 두 실행 결과가 일치한다.

26. 비교 결과를 남긴다

시간(ns)	a	b	x	y	z
0-10	0	0	0	0	0
10-20	0	1	0	1	1
20-30	1	0	0	1	1
30-40	1	1	1	1	0

VS Code 사전 로그: build/vscode/.

Vivado task 로그: build/vivado-sim/.

GUI 실행 로그: vivado/logic_gate.sim/sim_1/behav/xsim/.

경로와 결과를 구분해 보관한다.

27. 대상 보드와 핀 확인

Constraints → logic_gate.xdc. 이번 원본 장비의 매핑이다.

포트	PACKAGE_PIN	역할
a	K4	SW1
b	N8	SW2
x	L4	LED1 / AND
y	M4	LED2 / OR
z	M2	LED3 / XOR

IOSTANDARD는 모두 LVCMOS33. 원본 장비 XDC와 대조했다.
다른 보드의 핀 번호로 바꾸지 않는다. 이 회로는 클록이 없는 조합회로다.

28. 합성 → 구현 → 비트스트림

1. 시뮬레이션을 닫고 Flow Navigator의 Run Synthesis.
2. 완료 후 Run Implementation. 실패 메시지가 없는지 확인.
3. 완료 후 Generate Bitstream. 이전 단계가 필요하다는 안내가 뜨면 합성·구현부터 진행한다.
4. 완료 상태는 write_bitstream Complete!.

workspace의 06 Build bitstream은 위 순서를 자동 실행한다.
결과가 나올 때까지 기다린다. 단순히 작업을 시작한 화면은 성공 증거가 아니다.

29. 실제 비트스트림 생성 완료

Enabled

Location: C:/Users/UOS/Documents/ece2_26_2/example/fpga_projects

Type: SystemVerilog

General Properties

Tcl Console Messages Log Reports Design Runs

Q X Z |< << >> > + %

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												2	0
✓ impl_1	constrs_1	write_bitstream Complete!	NA	NA	NA	NA		NA	2.781	0				2	0

Synthesis

Status: ✓ Complete

Messages: No errors or warnings

Part: xc7s75fzga484-1

Implementation

Status: ✓ Complete

Messages: 3 warning

Part: xc7s75fzga484-1

Design Runs에서 synth_1과 impl_1의 완료 표시를 확인한다.
최종 상태는 write_bitstream Complete!
이 단계는 파일 생성이며 아직 FPGA에 기록한 상태는 아니다.

30. 생성 파일과 경고 읽기

- Vivado 원본: vivado/logic_gate.runs/impl_1/logic_gate.bit
- task가 복사하는 결과: build/build/design.bit
- DRC: CFGBVS-1 경고 1건. 구성 bank 전압 속성이 없어 실제 보드 회로도 확인 후 지정해야 한다.
- 클록이 없는 회로라 setup/hold 타이밍 표에 NA가 나올 수 있다. 이를 클록 회로의 타이밍 통과와 혼동하지 않는다.

핀 미지정 UCIO·전기 규격 미지정 NSTD 같은 오류를 무시하고 비트스트림을 만들지 않는다.

다음 실험에서는 Hardware Manager로 실제 장치에 기록하고 스위치·LED 동작을 촬영한다.

31. 실습 완료 점검

1. 템플릿 clone부터 workspace 열기까지 직접 실행했는가?
2. VS Code에서 파일·작업·로그·파형을 직접 확인했는가?
3. 두 시뮬레이션의 같은 코드·네 입력·기대값이 연결되는가?
4. Vivado의 소스 종류와 top, 핀, 비트스트림 결과가 구분되는가?

사전 레포트에 자신의 실행 결과와 해석을 정리한다.

실험 후 레포트에는 Vivado 결과와 실제 보드 기록·사진·영상·GitHub 링크를 추가한다.

▶ 첫 회로 소스·workspace·검증 안내